

Projet – pédagogie active

Inventer une variante d'un problème classique de concurrence

Du lore d'origine à une implémentation C++20 – en binôme

En une phrase

Vous recevez un problème classique de synchronisation, vous en étudiez la structure, vous la transposez dans un nouveau lore de votre invention, puis vous l'implémentez en C++20 en deux versions : l'une correcte, l'autre volontairement interbloquée.

Travail en
Volume
Évaluation
Dépôt

binômes (groupes de 2)
2 séances de 4h – soit 8 h encadrées
20 points répartis sur 5 livrables, bonus possibles
Cours AMeTICE "[26-27]-R5.05 Prog. concurrente"

Les problèmes étudiés sont issus de *The Little Book of Semaphores* d'Allen B. Downey (chapitre 5), disponible sur le cours AMeTICE ou au lien suivant : <https://greenteapress.com/semaphores/LittleBookOfSemaphores.pdf>.



1. Ce qu'on vous demande, et pourquoi

Les problèmes classiques de la programmation concurrente – les philosophes, les lecteurs et les rédacteurs, le producteur et le consommateur – ont deux caractéristiques utiles. Leur *lore*, c'est-à-dire l'histoire dans laquelle ils sont racontés, est arbitraire : les philosophes pourraient être des imprimantes, les fourchettes des jetons de licence. Mais la **structure** qui se cache derrière, elle, ne l'est pas : c'est elle qui rend le problème intéressant, et c'est elle qu'on retrouve dans les vrais systèmes.

Séparer les deux est précisément la compétence que ce projet évalue. Vous saurez que vous avez compris un problème de concurrence le jour où vous serez capable de le raconter autrement sans rien lui enlever.

Objectifs

- Identifier et expliquer les concepts de concurrence mobilisés par un problème donné.
- Distinguer ce qui est accessoire (le décor) de ce qui est essentiel (les contraintes de synchronisation).
- Concevoir une variante originale qui préserve cette structure.
- Implémenter, tester et documenter une solution en C++20, ainsi qu'une version volontairement fautive.
- Restituer le travail à l'oral, en dix minutes, devant la promotion.

1.1. Organisation

Le travail se fait en **binômes**. Chaque binôme travaille sur **un problème différent**, attribué avant la première séance. L'attribution résulte d'un **vote par classement** : chaque binôme ordonne les problèmes d'une présélection par ordre de préférence, et les attributions sont calculées à partir de ces classements.

Votre problème

La présélection est faite chaque année dans le **chapitre 5 du *Little Book of Semaphores*** d'Allen B. Downey (*Less classical synchronization problems*). Le chapitre vous est fourni. Il contient, pour chaque problème, l'énoncé, un indice et la solution de l'auteur en pseudo-code.

Liste retenue cette année : [\[liste des problèmes présélectionnés\]](#).

Modalités du vote : [\[date, support et règle de départage\]](#).

Une précision importante sur le livre

Downey donne la solution de chaque problème. Ce n'est pas un problème pour ce projet : votre travail ne consiste pas à *trouver* la solution, mais à *comprendre* pourquoi elle fonctionne, à l'abstraire, puis à la transposer. Une transposition mécanique du pseudo-code sans compréhension se voit immédiatement, que ce soit au livrable 2, à l'oral, ou aux questions.

2. Usage de l'intelligence artificielle générative

La seule règle non négociable

La **génération de code** par une IA générative est **interdite**. Transgresser cette règle mènera inéluctablement à une saisine du conseil de discipline, sans négociation possible.

Pour tout le reste, l'usage est autorisé, voire recommandé, mais de manière encadrée :

- **Brainstorming** : recommandé, à condition de s'en servir pour aller *plus loin* – explorer des

pistes, tester la solidité d'une idée, etc. – et non pour aller *plus vite*.

- **Rédaction des livrables** : autorisée, voire même recommandée pour le livrable 1 et pour les sections 1 et 2 du livrable 3. Dans tous les cas, vous êtes directement responsables de la qualité de la production, qui sera évaluée sur le fond comme sur la forme : un texte généré et non relu se repère, et il est noté comme tel.

Livrable 6 – Déclaration d'usage

La liste **honnête** des usages que vous avez faits de l'IA générative pendant le projet : à quelles étapes, pour quoi faire, et ce que vous en avez retenu ou écarté.

Ce livrable n'est pas noté. Il est demandé pour information, et son absence est prise en compte dans l'appréciation générale.

3. Déroulé

Le projet se déroule en cinq phases. Les quatre premières produisent chacune un livrable écrit ou logiciel, la cinquième une restitution orale.

Phase	Objet	Livrable	Échéance
1	Comprendre les concepts	L1 – glossaire	Voir AMeTICE
2	Extraire la structure	L2 – caractéristiques fondamentales	Voir AMeTICE
3	Changer de lore	L3 – nouveau problème, deadlock, tableau de correspondance	Voir AMeTICE
4	Implémenter	L4 – projet Qt Creator, deux versions, tests	Voir AMeTICE
5	Restituer	L5 – présentation orale et diapositives	Voir AMeTICE
–	Transparence	L6 – usages de l'IA générative	Voir AMeTICE

Découpage indicatif du temps encadré : 3 h pour les phases 1 à 3, 3 h pour la phase 4, 2 h pour la finalisation et la préparation de l'oral.

3.1. Phase 1 – Identifier et comprendre les concepts

Votre problème mobilise un certain nombre de concepts de programmation concurrente. Certains apparaissent explicitement dans l'énoncé, d'autres sont cachés dans la solution.

- Listez l'**ensemble** des concepts impliqués par le problème et par sa solution.
- Ajoutez-y la notion d'**interblocage** (*deadlock*), qui fait partie du glossaire quel que soit votre problème.
- Pour chacun, produisez une définition qui soit à la fois **juste** et **compréhensible** par un étudiant de BUT1 ou 2 qui n'a pas suivi ce module.

Livrable 1 – Glossaire des concepts

Un document listant chaque concept mobilisé, avec sa définition et, quand c'est utile, un exemple tiré de votre problème.

Format : **[format et nommage attendus]**.

Sur la vulgarisation

Une définition qui recopie bêtement le cours, Wikipédia ou la première réponse d'une IA n'est pas une définition comprise. La vulgarisation est un exercice difficile : une définition trop courte peut être incomplète et incompréhensible sans bagage supplémentaire ; et, à l'inverse, une définition trop longue peut perdre votre interlocuteur avec des détails superflus. Prenez le temps de faire l'exercice, et prenez du recul par rapport à vos productions.

3.2. Phase 2 – Extraire la structure du problème

C'est la phase la plus exigeante du projet, et celle qui détermine la qualité de tout ce qui suit.

1. **Listez les caractéristiques** de votre problème : les acteurs, les ressources, les contraintes de synchronisation, les conditions d'attente, les propriétés attendues.
2. **Triez-les** : lesquelles sont essentielles – c'est-à-dire que les supprimer changerait la nature du problème – et lesquelles relèvent du décor ?
3. **Généralisez** les caractéristiques essentielles : formulez-les sans employer aucun mot du lore d'origine.

Le test de la généralisation

Une caractéristique correctement généralisée ne contient plus ni barbier, ni marmite, ni renne.
« Le service ne peut commencer que si un client et un serveur sont simultanément disponibles » est généralisé ; « le barbier ne coupe que s'il y a un client » ne l'est pas.

Livrable 2 – Caractéristiques fondamentales généralisées

La liste des caractéristiques essentielles de votre problème, formulées indépendamment du cadre applicatif d'origine.

Format : [format et nommage attendus].

3.3. Phase 3 – Changer de lore

Vous inventez maintenant un nouveau cadre applicatif pour la structure que vous venez d'isoler.

- **Définissez le nouveau problème** : qui sont les acteurs, quelles sont les ressources, que cherche-t-on à faire ?
- **Identifiez une situation d'interblocage** possible dans ce nouveau cadre, et **démontrez** qu'il s'agit bien d'un interblocage.
- **Montrez que les caractéristiques fondamentales sont préservées**, une par une.

Blocage n'est pas interblocage

Un programme qui ne progresse plus n'est pas nécessairement en situation d'interblocage. Un interblocage est un état où un ensemble de tâches s'attendent mutuellement, sans qu'aucune ne puisse jamais progresser. Pour le démontrer, exhibez le scénario complet et montrez que les **quatre conditions de Coffman** sont réunies.

Livrable 3 – Le nouveau problème

Un document en trois sections :

1. description du nouveau problème ;
2. description de l'interblocage, avec la preuve qu'il s'agit bien d'un interblocage ;
3. tableau à trois colonnes : chaque caractéristique fondamentale du livrable 2, sa déclinaison dans le lore d'origine, sa déclinaison dans le nouveau lore.

Format : [format et nommage attendus].

3.4. Phase 4 – Implémenter

- Implémentez une **solution correcte** de votre nouveau problème, sans interblocage.
- **Clonez-la** et introduisez-y l'interblocage décrit au livrable 3.
- Écrivez des **tests** qui prouvent le comportement de chacune des deux versions : au moins un pour la version correcte, au moins un pour la version fautive.

- **Commentez** le code et **expliquez les tests** : leur(s) objectif(s) et les résultats / observations attendu(e)s.

Langage et outils : **C++20**, sous **Qt Creator**. Vous disposez de tout ce qui a été vu en cours et en TP : `std::jthread`, `std::mutex` et les verrous RAII, `std::atomic`, `std::counting_semaphore`, `std::condition_variable`. Le choix des bons outils fait partie du travail et sera discuté à l'oral.

Comment teste-t-on une version qui se bloque ?

C'est la bonne question, et elle se pose dans les deux sens. Pour la version correcte, il faut montrer que le programme termine, même dans des conditions difficiles/particulières, *et* que l'invariant est respecté. Pour la version fautive, un test qui attend indéfiniment n'est pas un bon test : utilisez des `print` et la console pour montrer que vous aboutissez bien à un interblocage.

Une exécution ne prouve rien

Un bug de concurrence peut ne se manifester qu'une fois sur mille. Un test qui passe une fois ne démontre rien : répétez, faites varier le nombre de threads, et dites honnêtement dans vos commentaires ce que votre test établit et ce qu'il n'établit pas.

Livrable 4 – Projet Qt Creator

Un projet contenant les deux implémentations et les tests, **commentés**, ainsi que les résultats d'exécution attendus. L'organisation du code est libre : un fichier unique convient si le code reste court et lisible, sinon découpez.

Format : [format d'archive, nommage et modalités de dépôt].

3.5. Phase 5 – Restituer

Présentation devant la promotion : **10 minutes au total**, dont **6 minutes d'exposé** et **4 minutes de questions**. La structure suivante est imposée.

1. Introduire le problème d'origine avec son lore.
2. Présenter les concepts de concurrence sous-jacents : un seul concept central est développé – **déterminé avec l'enseignant** – les autres sont simplement mentionnés.
3. Décrire le nouveau lore.
4. Présenter l'interblocage imaginé.
5. Mentionner les outils C++20 utilisés.
6. Conclure par une **unique** diapositive de synthèse.

Six minutes, c'est court

Six points à couvrir en six minutes : chacun tient en une diapositive et en plus ou moins une minute. **C'est un exercice de synthèse**. Les détails sont couverts par les autres livrables intermédiaires. Répétez avec un chronomètre. Si vous dépassez, **vous serez coupés** sans sommation.

Livrable 5 – Présentation

Les diapositives, déposées avant l'oral, l'exposé lui-même et les réponses aux questions.

Format et échéance de dépôt : [à préciser].

4. Évaluation

Le total est de 20 points. Certains critères permettent de dépasser l'enveloppe annoncée : ce sont des **bonus**.

Livrable	Critère	Points
L1 – Glossaire (/2,5)	Tous les concepts sont identifiés : non (0) / oui	1
	Qualité des définitions : non rendu ou trop mauvaise (0), qualité variable, manque de clarté, phrases vides de sens (0,5 à 1), claires et traduisant une bonne compréhension	1,5
L2 – Caractéristiques (/2)	Toutes les caractéristiques fondamentales importantes sont identifiées : non (0) / oui	1
	Qualité de la généralisation : pas fait ou trop mauvaise (0), passable (0,5), bonne	1
L3 – Nouveau lore (/2,5)	Qualité du nouveau lore : pas fait (0), pas totalement adapté ou description peu claire (0,5 à 1), préserve l'ensemble des caractéristiques fondamentales	1,5
	Qualité de l'interblocage : pas fait, hors sujet ou vide de sens (0), blocage mais pas interblocage (0,25), correct	1
	<i>Bonus</i> originalité et créativité	+0,5
L4 – Code (/7)	Code qui compile : version sans interblocage (1), version avec interblocage (1)	2
	Tests de la version sans interblocage : pas faits (0), insuffisants (0,5), suffisants	1
	Tests de la version avec interblocage : pas faits (0), insuffisants (0,5), suffisants	1
	Qualité du code – nommage, découpage en fonctions, organisation : non rendu ou catastrophique (0), passable (0,5 à 1,5), bonne	2
	Qualité des commentaires : aucun (0), passable – inutiles, vides de sens, trop peu nombreux (0,5), facile à lire et à prendre en main	1
L5 – Oral (/6)	Respect de la structure imposée : non ou pas fait (0), oui	1
	Esprit de synthèse : pas fait ou très mauvais (0), insuffisant dans l'ensemble (0,5), insuffisant sur certaines parties (1), bon dans l'ensemble	1,5
	Qualité des diapositives : non rendu ou catastrophique (0), forme soignée avec trop de fautes (0,5), forme soignée sans faute	1
	Réponses aux questions : pas fait ou très insuffisant (0), insuffisant ou trop peu clair (0,5), suffisant (1,5), très bien	2
	Respect du temps de parole : non ou pas fait (0), mauvaise répartition (0,5), oui	1
L6	Déclaration d'usage de l'IA	non noté

Remarque

Le code pèse 7 points sur les 20. Prenez le temps de réfléchir avant de vous lancer, et de prendre du recul sur la qualité de vos productions à chaque étape avant de passer à la suite. Vous êtes en troisième année maintenant : montrez que vous êtes plus que des cracheurs de code. Toutes les compétences travaillées dans ce projet vous seront utiles dans votre futur professionnel, voire même personnel.

5. Calendrier et modalités pratiques

Séance	Date	Objet
Avant S1	Semaine du 28/09	Vote par classement, attribution des problèmes
Séance 1	Semaine du 05/10	Phases 1 à 3 – analyse
Séance 2	Semaine du 12/10	Phase 4 – implémentation, finalisation, préparation de l'oral
Soutenances	Semaine du 19/10	Phase 5 – restitutions

Dépôt. Cours AMeTICE "[26-27]-R5.05 Prog. concurrente". *Les deadlines par livrable sont fixées là-bas.*

Retards. 0 pour le livrable.

Référence : Allen B. Downey, *The Little Book of Semaphores*, Green Tea Press, chapitre 5 *Less classical synchronization problems*. Ouvrage diffusé sous licence libre.

Annexe – notes à l’attention des enseignants

Cette annexe ne figure pas dans la version distribuée aux étudiants. Pour l’obtenir, basculer le commutateur du préambule sur :

`\annexeenseignantefalse`

Ce que le projet évalue réellement

Le cœur de l’évaluation est le livrable 2. C’est le seul endroit où l’on voit si le binôme a compris la structure du problème ou s’il en a recopié la solution. Un tableau de correspondance (livrable 3, section 3) dont la colonne « caractéristique généralisée » contient encore du vocabulaire du lore d’origine est le signal le plus fiable d’une compréhension superficielle – il se repère en trente secondes et permet de recadrer tôt.

Calibrage de la présélection

Les problèmes du chapitre 5 ne sont pas de difficulté comparable. Points de vigilance à l’attribution :

- *Hilzer’s barbershop* et le *grand huit multi-voitures* sont nettement plus lourds que les autres – files de sémaphores, sémaphore dont le propriétaire change, plusieurs rendez-vous imbriqués.
- *H₂O* et la *traversée de la rivière* reposent sur une barrière, qui n’est pas introduite en CI2. À réserver à des binômes solides, ou à accompagner.
- *Dining savages* porte un lore colonial dont l’auteur s’excuse lui-même en note. Le changement de lore résout le problème, mais il vaut mieux l’annoncer, ou écarter le sujet.
- *Barbershop*, *FIFO barbershop*, *Santa Claus* et le *grand huit* à une voiture constituent le cœur de gamme accessible.

[Décision de l’équipe pédagogique pour cette année]

Points de passage du rôle de « client »

Trois arbitrages qui gagnent à être traités en séance plutôt que découverts à la correction :

1. **Valider le livrable 2 avant que le binôme ne code.** Un lore validé tardivement produit une implémentation à refaire.
2. **Choisir avec eux le concept central de l’oral** (phase 5, point 2). Sans cet arbitrage, ils survolent tout et ne développent rien.